

Modulo 2

un programador escribe un programa y el programa hace respuesta, una computadora ejecuta el programa y proporciona las respuestas
el programa debe ser capaz de reaccionar de acuerdo con las respuestas recibidas.

Tabla de prioridades:

+ , - Unario
* , *
/ , / , // ,
+ , -
< , <= , > , >=
= , ! , !=

El bucle while ejecuta un sentencia o un conjunto de sentencias siempre que una expresión booleana específica sea verdadera.

El bucle for ejecuta un conjunto de sentencias muchas veces se usa para iterar un secuencia.

La cláusula else se ejecuta después de que el bucle termina su ejecución siempre y cuando no hay terminado con el break.

La función range() genera una secuencia de números enteros y devuelve objetos de rango opera dos lógicos.

AND: Si ambos operandos son verdaderos la condición es verdadera o si alguno de los operandos es verdadero la condición es verdadera.

NOT: Devuelve falso si el estado es verdadero y devuelve true si es falso.

La lista es un tipo de dato en python que se utiliza para almacenar información y múltiples objetos es una colección ordenada y mutable de elementos separados por comas entre corchetes.

El método `sort()` se utiliza para ordenar los elementos de una lista.

El método `reverse()` se usa para invertir una lista. Si tienes una lista `list-1` y la siguiente es `list-2` quite a la misma lista en una memoria.

La flexibilidad de las listas te permite crear nuevas listas a partir de las existentes de una manera rápida y flexible puedes usar listas anidadas en python para crear matrices es decir, listas bidimensionales.

Por ejemplo

`y = 0`

```
1 0 1 0 1
2 1 0 1 0
3 0 1 0 1
x 0 1 2 3
```

Puedes anidar tantas listas en las listas como desees, por lo tanto crear listas de n -dimensionales, por ejemplo crear 4 tess, y o incluso sesenta, y cuatro dimensiones.



PythonINSTITUTE

3.1.1.10 LABORATORIO: Operadores de comparación y ejecución condicional

MODULE (15%)

SECTION (75%)

LABORATORIO

10-15 minutos

Fácil/Medio

Objetivos

Familiarizar al estudiante con:

- Imprimir el enunciado "Si, ¡El ESPATIFILIO! es la mejor planta de todos los tiempos!" en la pantalla si la cadena ingresada es "ESPATIFILIO".
- Imprimir "No, ¡quiero un gran ESPATIFILIO!" si la cadena ingresada es "espatifilo".
- Imprimir "¡ESPATIFILIO!, ¡No [entrada]!" de lo contrario. Nota: [entrada] es la cadena que se toma como entrada.

Prueba tu código con los datos que te proporcionamos. ¡Y hazte de un ESPATIFILIO también!

Datos de Prueba

Entrada de muestra: `espatifilo`

Resultado esperado: `No, ¡quiero un gran ESPATIFILIO!`

Entrada de ejemplo: `pelargonio`

Resultado esperado: `¡ESPATIFILIO!, ¡No pelargonio!`

Entrada de muestra: `ESPATIFILIO`

Resultado esperado: `Si, ¡El ESPATIFILIO es la mejor planta de todos los tiempos!`

```
1 planta = input()
2
3 if planta == "ESPATIFILIO":
4     print("Si, ¡El ESPATIFILIO es la mejor planta de todos los tiempos!")
5 elif planta == "espatifilo":
6     print("No, ¡quiero un gran ESPATIFILIO!")
7 else:
8     print("¡ESPATIFILIO!, ¡No [planta]!")
9
```

Console >...

espatifilo
No, ¡quiero un gran ESPATIFILIO!

R

Ricardo Daniel
1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

3.1.1.11 LABORATORIO: Fundamentos de la sentencia if-else

MODULE (15%)

SECTION (85%)

LABORATORIO

10-15 minutos

Fácil/Medio

Objetivos

Familiarizar al estudiante con:

- Utilizar la sentencia *if-else* para ramificar la ruta de control.
- Construir un programa completo que resuelva problemas simples de la vida real.

Escenario

Érase una vez una tierra de leche y miel, habitada por gente feliz y próspera. La gente pagaba impuestos, por supuesto, su felicidad tenía límites. El impuesto más importante, denominado *Impuesto Personal de Ingresos* (*IPI*, para abreviar) tenía que pagarse una vez al año y se evaluó utilizando la siguiente regla:

- Si el ingreso del ciudadano no era superior a 85,528 pesos, el impuesto era igual al 18% del ingreso menos 558 pesos y 2 centavos (esta fue la llamada exención fiscal).
- Si el ingreso era superior a esta cantidad, el impuesto era igual a 14,839 pesos y 2 centavos, más el 32% del excedente sobre 85,528 pesos.

Tu tarea es escribir una **calculadora de impuestos**.

- Debe aceptar un valor de punto flotante: el ingreso.
- A continuación, debe imprimir el impuesto calculado, redondeado a pesos totales. Hay una función llamada `round()` que hará el redondeo por ti, la encontrarás en el código de esqueleto del editor.

Nota: Este país feliz nunca devuelve dinero a sus ciudadanos. Si el impuesto calculado es menor que cero, solo significa que no hay impuesto (el impuesto es igual a cero). Ten esto en cuenta durante tus cálculos.

```
1 income = float(input("Introduce el ingreso anual: "))
2
3 # Tu código aquí.
4 if income <= 85528:
5     tax = (income * 0.18) - 556.02
6 else:
7     tax = 14839.02 + ((income - 85528) * 0.32)
8
9 # Si el impuesto calculado es menor que cero, es igual a cero
10 if tax < 0:
11     tax = 0.0
12
13 tax = round(tax, 0)
14 print("El impuesto es:", tax, "pesos")
15
```

Console >...

Introduce el ingreso anual: 1000
El impuesto es: 0.0 pesos

R

Ricardo Daniel
1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

3.1.1.12 LABORATORIO: Fundamentos de la sentencia if-elif-else

MODULE (15%)

SECTION (85%)

LABORATORIO

10-15 minutos

Fácil/Medio

Objetivos

Familiarizar al estudiante con:

- Utilizar la sentencia *if-elif-else*.
- Encontrar la implementación adecuada de las reglas definidas verbalmente.
- Emplear el código de prueba empleando entradas y salidas de muestra.

Escenario

Como seguramente sabrás, debido a algunas razones astronómicas, el año pueden ser *bisiesto* o común. Los primeros tienen una duración de 366 días, mientras que los últimos tienen una duración de 365 días.

Desde la introducción del calendario Gregoriano (en 1582), se utiliza la siguiente regla para determinar el tipo de año:

- Si el número del año no es divisible entre cuatro, es un *año común*.
- De lo contrario, si el número del año no es divisible entre 100, es un *año bisiesto*.
- De lo contrario, si el número del año no es divisible entre 400, es un *año común*.
- De lo contrario, es un *año bisiesto*.

Observa el código en el editor: solo lee un número de año y debe completarse con las instrucciones que implementan la prueba que acabamos de describir.

El código debe mostrar uno de los dos mensajes posibles, que son `Año Bisiesto` o `Año Común`, según el valor ingresado.

```
1 year = int(input("Introduce un año:"))
2
3 #
4 # Escribe tu código aquí.
5 #
6 year = int(input("Introduce un año: "))
7
8 if year % 4 != 0:
9     print("Año común")
10 elif year % 100 != 0:
11     print("Año bisiesto")
12 elif year % 400 != 0:
13     print("Año común")
14 else:
15     print("Año bisiesto")
16
17
```

Console >...

Introduce un año:2026
Introduce un año: 2000
Año bisiesto
Introduce un año:2025
Introduce un año: 2015
Año común

PythonINSTITUTE

3.2.1.3 LABORATORIO: Lo esencial del bucle while - Adivina el número secreto »

MODULE (24%)

SECTION (18%)

LABORATORIO

Tiempo Estimado

15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Utilizar el bucle `while`.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

Un mago junior ha elegido un número secreto. Lo ha escondido en una variable llamada `secret_number`. Quiere que todos los que ejecutan su programa jueguen el juego *Adivina el número secreto*, y adivina qué número ha elegido para ellos. ¡Quiénes no adivinen el número quedarán atrapados en un bucle sin fin para siempre! Desafortunadamente, él no sabe cómo completar el código.

Tu tarea es ayudar al mago a completar el código en el editor de tal manera que el código:

- Pedirá al usuario que ingrese un número entero.
- Utilizará un bucle `while`.
- Comprobará si el número ingresado por el usuario es el mismo que el número escogido por el mago. Si el número elegido por el usuario es diferente al número secreto del mago, el usuario debería ver el mensaje `"¡Ja, ja! ¡Estás atrapado en mi bucle!"` y se le solicitará que ingrese un número nuevamente. Si el número ingresado por el usuario coincide con el número escogido por el mago, el número debe imprimirse en la pantalla, y el mago debe decir las siguientes palabras: `"¡Bien hecho, muggle! Eres libre ahora"`.

```
1 secret_number = 777
2
3 print(
4     """
5     =====
6     | Bienvenido a mi juego, muggle!  |
7     | Introdúcelo un número entero   |
8     | y adivina qué número he       |
9     | elegido para ti.               |
10    | Entonces,                      |
11    | ¿Cuál es el número secreto?    |
12    | =====
13    |
14    |
15    |
16    numero = int(input())
17
18 while numero != secret_number:
19     print("¡Ja, ja! ¡Estás atrapado en mi bucle!")
20     numero = int(input())
21
22 print("¡Bien hecho, muggle! Eres libre ahora")
```

Console >_

```
=====
| Bienvenido a mi juego, muggle! |
| Introdúcelo un número entero   |
| y adivina qué número he       |
| elegido para ti.               |
| Entonces,                      |
| ¿Cuál es el número secreto?    |
| =====
10
¡Ja, ja! ¡Estás atrapado en mi bucle!
777
¡Bien hecho, muggle! Eres libre ahora
```

Ricardo Daniel
1225100759.rdi@gmail.com

Inicio sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

3.2.1.6 LABORATORIO: Fundamentos del bucle for: el conteo »

MODULE (28%)

SECTION (33%)

LABORATORIO

Tiempo Estimado

5 minutos

Nivel de dificultad

Muy fácil

Objetivos

Familiarizar al estudiante con:

- Utilizar el bucle `for`.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

¿Sabes lo que es Mississippi? Bueno, es el nombre de uno de los estados y ríos en los Estados Unidos. El río Mississippi tiene aproximadamente 2,340 millas de largo, lo que lo convierte en el segundo río más largo de los Estados Unidos (el más largo es el río Missouri). ¡Es tan largo que una sola gota de agua necesita 90 días para recorrer toda su longitud!

La palabra *Mississippi* también se usa para un propósito ligeramente diferente: para contar mississippily (mississippimenter).

Si no estás familiarizado con la frase, estamos aquí para explicarte lo que significa: se utiliza para contar segundos.

La idea detrás de esto es que agregar la palabra *Mississippi* a un número al contar los segundos en voz alta hace que suene más cercano al reloj, y por lo tanto "un Mississippi, dos Mississippi, tres Mississippi" tomará aproximadamente unos tres segundos reales de tiempo. A menudo lo usan los niños que juegan al escondite para asegurarse de que el buscador haga un conteo honesto.

```
1 import time
2
3 # Escribe un bucle for que cuente hasta cinco.
4 # Cuerpo del bucle: imprime el número de iteración del bucle y la palabra "Mississippi"
5 # Cuerpo del bucle - usar: time.sleep (1)
6
7 # Escribe una función de impresión con el mensaje final.
8
9
10 for i in range(1, 6):
11     print(i, "Mississippi")
12     time.sleep(1)
13
14 print("¡Listos o no, ahí voy!")
```

Console >_

```
1 Mississippi
2 Mississippi
3 Mississippi
4 Mississippi
5 Mississippi
¡Listos o no, ahí voy!
```

Ricardo Daniel
1225100759.rdi@gmail.com

Inicio sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

3.2.1.9 LABORATORIO: La sentencia break - Atascado en un bucle »

MODULE (32%)

SECTION (33%)

LABORATORIO

Tiempo Estimado

10 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Utilizar la instrucción `break` en los bucles.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

La instrucción `break` se implementa para salir/terminar un bucle.

Diseña un programa que use un bucle `while` y le pida continuamente al usuario que ingrese una palabra a menos que ingrese "chupacabra" como la palabra de salida secreta, en cuyo caso el mensaje: `"¡Has dejado el bucle con éxito"`. Debe imprimirse en la pantalla y el bucle debe terminar.

No imprimas ninguna de las palabras ingresadas por el usuario. Utiliza el concepto de ejecución condicional y la sentencia `break`.

```
1 while True:
2     palabra = input("Ingresa una palabra: ")
3     if palabra == "chupacabra":
4         print("¡Has dejado el bucle con éxito.")
5         break
```

Console >_

```
Ingresa una palabra: chupacabra
¡Has dejado el bucle con éxito.
```

Ricardo Daniel
1225100759.rdi@gmail.com

Inicio sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

3.2.1.10 LABORATORIO: La sentencia continue - El Feo Devorador de Vocales

MODULE (33%)

SECTION (58%)

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Utilizar la instrucción `continue` en los bucles.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

La sentencia `continue` se usa para omitir el bloque actual y avanzar a la siguiente iteración, sin ejecutar las sentencias dentro del bucle.

Se puede usar tanto con `while` y `for`.

Tu tarea aquí es muy especial: ¡Debes diseñar un devorador de vocales! Escribe un programa que use:

- Un bucle `for`.
- El concepto de ejecución condicional (`if-elif-else`).
- La sentencia `continue`.

Tu programa debe:

- Pedir al usuario que ingrese una palabra.
- Utiliza `user_word = user_word.upper()` para convertir la palabra ingresada por el usuario a mayúsculas; hablaremos sobre los llamados **métodos de cadena** y el `upper()` muy pronto, no te preocupes.

```
1 # Indicar al usuario que ingrese una palabra
2 user_word = input("Ingresar una palabra:")
3 # Y asignarlo a la variable user_word convirtiéndola a mayúsculas
4 user_word = user_word.upper()
5
6 for letter in user_word:
7     # Completa el cuerpo del bucle for.
8     if letter == "a":
9         continue
10    elif letter == "e":
11        continue
12    elif letter == "i":
13        continue
14    elif letter == "o":
15        continue
16    elif letter == "u":
17        continue
18    else:
19        print(letter)
```

Console >_

Ingresar una palabra: Gregory

G

R

G

R

Y

Ricardo Daniel

1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

3.2.1.11 LABORATORIO: La sentencia continue - El Bonito Devorador de Vocales

MODULE (35%)

SECTION (58%)

LABORATORIO

Tiempo Estimado

5-10 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Utilizar la instrucción `continue` en los bucles.
- Modificar y actualizar el código existente.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

Tu tarea aquí es aún más especial que antes: ¡Debes rediseñar el devorador de vocales (feo) del laboratorio anterior (3.1.2.10) y crear un mejor devorador de vocales (bonito) mejorado! Escribe un programa que use:

- Un bucle `for`.
- El concepto de ejecución condicional (`if-elif-else`).
- La instrucción `continue`.

Tu programa debe:

- Pedir al usuario que ingrese una palabra.
- Utilizar `user_word = user_word.upper()` para convertir la palabra ingresada por el usuario a mayúsculas; hablaremos sobre los llamados **métodos de cadena** y el `upper()` muy pronto, no te preocupes.
- Emplea la ejecución condicional y la instrucción `continue` para "comer" las siguientes vocales `A`, `E`, `I`, `O`, `U` de la palabra ingresada.
- Asigna las letras no consumidas a la variable `word_without_vowels` e imprime la variable en la consola.

```
1 word_without_vowels = ""
2
3 # Indicar al usuario que ingrese una palabra
4 user_word = input("Ingresar una palabra:")
5 # Y asignarla a la variable user_word convirtiéndola a mayúsculas
6 user_word = user_word.upper()
7
8 for letter in user_word:
9     # Completa el cuerpo del bucle.
10    if letter == "A":
11        continue
12    elif letter == "E":
13        continue
14    elif letter == "I":
15        continue
16    elif letter == "O":
17        continue
18    elif letter == "U":
19        continue
20    else:
21        word_without_vowels += letter
22
23 # Imprimir la palabra asignada a word_without_vowels.
24 print(word_without_vowels)
```

Console >_

Ingresar una palabra: Gregory

GRGRY

Ricardo Daniel

1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

3.2.1.14 LABORATORIO: Fundamentos del bucle while

MODULE (39%)

SECTION (82%)

LABORATORIO

Tiempo Estimado

20-30 minutos

Nivel de Dificultad

Medio

Objetivos

Familiarizar al estudiante con:

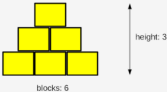
- Utilizar el bucle `while`.
- Encontrar la implementación adecuada de reglas definidas verbalmente.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

Escucha esta historia: Un niño y su padre, un programador de computadoras, juegan con bloques de madera. Están construyendo una pirámide.

Su pirámide es un poco rara, ya que en realidad es una pared en forma de pirámide, es plana. La pirámide se apila de acuerdo con un principio simple: cada capa inferior contiene un bloque más que la capa superior.

La figura ilustra la regla utilizada por los constructores:



```
1 blocks = int(input("Ingresar el número de bloques: "))
2
3 height = 0
4 blocks_needed = 1
5
6 while blocks >= blocks_needed:
7     blocks -= blocks_needed
8     height += 1
9     blocks_needed += 1
10
11 print("La altura de la pirámide:", height)
```

Console >_

Ingresar el número de bloques: 6

La altura de la pirámide: 3

Ricardo Daniel

1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

3.2.1.15 LABORATORIO: Hipótesis de Collatz

MODULE (40%)

SECTION (80%)

☰

★

⚙️

▶

🔍

📄

⏪

⏩

🔄

Escribe un programa que lea un número natural y ejecute los pasos anteriores siempre que `c0` sea diferente de 1. También queremos que cuente los pasos necesarios para lograr el objetivo. Tu código también debe mostrar todos los valores intermedios de `c0`.

Sugerencia: la parte más importante del problema es como transformar la idea de Collatz en un bucle `while` - esta es la clave del éxito.

Prueba tu código con los datos que hemos proporcionado.

Test Data

Entrada de muestra: 15

Salida esperada:

```
46
23
70
35
106
53
160
80
40
20
10
5
16
8
4
2
1
pasos = 17
```

```
1 c0 = int(input("Ingresa un número natural: "))
2
3 pasos = 0
4
5 while c0 != 1:
6     if c0 % 2 == 0:
7         c0 = c0 // 2
8     else:
9         c0 = 3 * c0 + 1
10    print(c0)
11    pasos += 1
12
13 print("pasos =", pasos)
```

Console >...

```
35
106
53
160
80
40
20
10
5
16
8
4
2
1
pasos = 17
```

Ricardo Daniel

1225100759.rdi@gmail.com

📄

Iniciar sesión en tu teléfono

🔑

Contraseñas y Autocompletar

🔑

Gestionar tu cuenta de Google

👤

Personalizar perfil

🔄

La sincronización está activada

➤

Añadir perfil de Chrome

🔑

Perfil de invitado abierto

➤

Gestionar perfiles de Chrome

PythonINSTITUTE

3.4.1.6 LABORATORIO: Lo básico de las listas

MODULE (50%)

SECTION (43%)

☰

★

⚙️

▶

🔍

📄

⏪

⏩

🔄

LABORATORIO

Tiempo Estimado

5 minutos

Nivel de Dificultad

Muy fácil

Objetivos

Familiarizar al estudiante con:

- Usar instrucciones básicas relacionadas con listas.
- Crear y modificar listas.

Escenario

Había una vez un sombrero. El sombrero no contenía conejo, sino una lista de cinco números: 1, 2, 3, 4 y 5.

Tu tarea es:

- Escribir una línea de código que solicite al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario (Paso 1).
- Escribir una línea de código que elimine el último elemento de la lista (Paso 2).
- Escribir una línea de código que imprima la longitud de la lista existente (Paso 3).

¿Listo para este desafío?

```
1 hat_list = [1, 2, 3, 4, 5] # Esta es una lista existente de números ocultos en el sombrero.
2
3 # Paso 1: escribe una línea de código que solicite al usuario
4 # reemplazar el número de en medio con un número entero ingresado por el usuario.
5 hat_list[2] = int(input("Ingresa un número entero para el centro: "))
6
7 # Paso 2: escribe aquí una línea de código que elimine el último elemento de la lista.
8 del hat_list[-1] # o también puedes usar hat_list.pop()
9
10 # Paso 3: escribe aquí una línea de código que imprima la longitud de la lista existente
11 print(len(hat_list))
12
13 print(hat_list)
```

Console >...

```
Ingresa un número entero para el centro: 20
4
[1, 2, 20, 4]
```

Ricardo Daniel

1225100759.rdi@gmail.com

📄

Iniciar sesión en tu teléfono

🔑

Contraseñas y Autocompletar

🔑

Gestionar tu cuenta de Google

👤

Personalizar perfil

🔄

La sincronización está activada

➤

Añadir perfil de Chrome

🔑

Perfil de invitado abierto

➤

Gestionar perfiles de Chrome

PythonINSTITUTE

3.4.1.13 LABORATORIO: Lo básico de las listas - Los Beatles

MODULE (68%)

SECTION (93%)

☰

★

⚙️

▶

🔍

📄

⏪

⏩

🔄

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Crear y modificar listas simples.
- Utilizar métodos para modificar listas.

Escenario

Los Beatles fueron uno de los grupos de música más populares de la década de 1960 y la banda más vendida en la historia. Algunas personas los consideran el acto más influyente de la era del rock. De hecho, se incluyeron en la compilación de la revista *Time* de las 100 personas más influyentes del siglo XX.

a banda sufrió muchos cambios de formación, que culminaron en 1962 con la formación de John Lennon, Paul McCartney, George Harrison y Richard Starkey (mejor conocido como Ringo Starr).

Escribe un programa que refleje estos cambios y le permita practicar con el concepto de listas. Tu tarea es:

- Paso 1: Crea una lista vacía llamada `beatles`.
- Paso 2: Emplea el método `append()` para agregar los siguientes miembros de la banda a la lista: John Lennon, Paul McCartney y George Harrison.
- Paso 3: Emplea el bucle `for` y el `append()` para pedirle al usuario que agregue los siguientes miembros de la banda a la lista: Stu Sutcliffe y Pete Best.
- Paso 4: Usa la instrucción `del` para eliminar a Stu Sutcliffe y Pete Best de la lista.
- Paso 5: Usa el método `insert()` para agregar a Ringo Starr al principio de la lista.

```
1 # paso 1
2 beatles = []
3 print("Paso 1:", beatles)
4
5 # paso 2
6 beatles.append("John Lennon")
7 beatles.append("Paul McCartney")
8 beatles.append("George Harrison")
9 print("Paso 2:", beatles)
10
11 # paso 3
12 for i in range(2):
13     beatles.append(input("Ingresa un miembro de la banda: "))
14 print("Paso 3:", beatles)
15
16 # paso 4
17 del beatles[-1]
18 del beatles[-1]
19 print("Paso 4:", beatles)
20
21 # paso 5
22 beatles.insert(0, "Ringo Starr")
23 print("Paso 5:", beatles)
24
25 # probando la longitud de la lista
26 print("¡Go! Fav", len(beatles))
```

Console >...

```
Paso 1: []
Paso 2: ['John Lennon', 'Paul McCartney', 'George Harrison']
Ingresa un miembro de la banda: John Lennon
Ingresa un miembro de la banda:
```

Ricardo Daniel

1225100759.rdi@gmail.com

📄

Iniciar sesión en tu teléfono

🔑

Contraseñas y Autocompletar

🔑

Gestionar tu cuenta de Google

👤

Personalizar perfil

🔄

La sincronización está activada

➤

Añadir perfil de Chrome

🔑

Perfil de invitado abierto

➤

Gestionar perfiles de Chrome



Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Indexación de listas.
- Utilizar operadores `in` y `not in`.

Escenario

Imagina una lista: no muy larga ni muy complicada, solo una lista simple que contiene algunos números enteros. Algunos de estos números pueden estar repetidos, y esta es la clave. No queremos ninguna repetición. Queremos que sean eliminados.

Tu tarea es escribir un programa que elimine todas las repeticiones de números de la lista. El objetivo es tener una lista en la que todos los números aparezcan no más de una vez.

Nota: Asume que la lista original está ya dentro del código, no tienes que ingresarla desde el teclado. Por supuesto, puedes mejorar el código y agregar una parte que pueda llevar a cabo una conversación con el usuario y obtener todos los datos.

Sugerencia: Te recomendamos que crees una nueva lista como área de trabajo temporal, no necesitas actualizar la lista actual.

No hemos proporcionado datos de prueba, ya que sería demasiado fácil. Puedes usar nuestro esqueleto en su lugar.

```
1 my_list = [1, 2, 4, 4, 1, 4, 2, 6, 2, 9]
2
3
4 # Creamos una lista temporal para guardar solo los elementos únicos
5 new_list = []
6
7 for number in my_list:
8     # Usamos 'not in' para verificar si el número ya está en la lista nueva
9     if number not in new_list:
10         new_list.append(number)
11
12 # Asignamos la lista única de vuelta a my_list
13 my_list = new_list
14
15 print("La lista con elementos únicos:")
16 print(my_list)
```

Console >_

```
La lista con elementos únicos:
[1, 2, 4, 6, 9]
```



Ricardo Daniel
1225100759rdi@gmail.com

- Iniciar sesión en tu teléfono
- Contraseñas y Autocompletar
- Gestionar tu cuenta de Google
- Personalizar perfil
- La sincronización está activada

- Añadir perfil de Chrome
- Perfil de invitado abierto
- Gestionar perfiles de Chrome